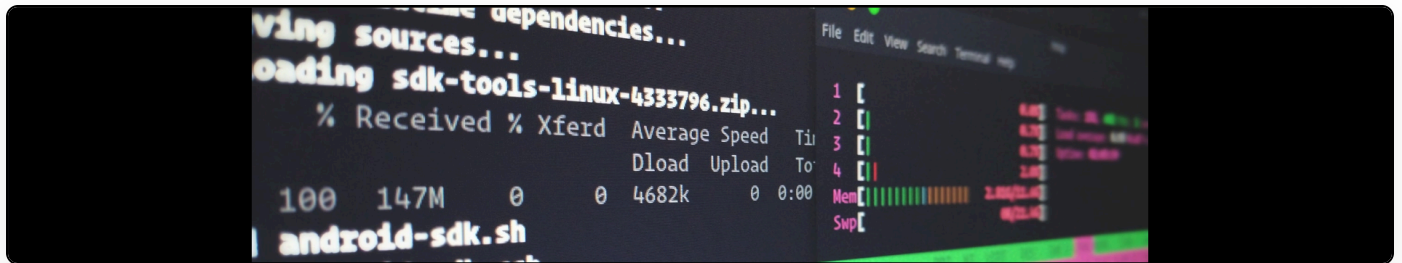


C# MySql DataExecutor class

Cite as: Martin Paul Eve, "C# MySql DataExecutor class", *eve.gd: Martin Paul Eve*, June 25, 2007, <https://doi.org/10.59348/z8wg9-eqf54>.



DataExecutor.cs:

```

//
// DataExecutor.cs
//
// Authors:
//     Martin Eve (martin@2bitpie.net)
//
//
// THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
// EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
// MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
// NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE
// LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION
// OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION
// WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
using System;
using System.Data;
using System.Configuration;
using MySql.Data.MySqlClient;
using System.Web.Security;
namespace Tools
{
    ///
    <summary>
        /// Provides a means of executing commands on the database
        /// </summary>
        /// <remarks>You must set the ConnectionString property in Web.Config:
        /// connectionStrings
        /// add name="ConnString" connectionString="Host=localhost;Database=DB;User
ID=xxxx;Password=xxx;"
        /// /connectionStrings
        /// </remarks>
    public class DataExecutor
    {
        public static bool testMode = false;
        public static DataExecutor testExecutor = null;
        public static string str_databaseConnect = string.Empty;
        private MySqlConnection Connection;
        public MySqlConnection TheConnection
        {
            get
            {
                if (!testMode)
                {
                    return Connection;
                }
                else
                {

```

```

        return new MySqlConnection();
    }
}
set { Connection = value; }
}
private MySqlCommand Command;
private MySqlDataAdapter DataAdapter;
private DataSet DS;
private int NonReader;
public delegate void TestModeFillInterceptor(ref DataTable dt);
public event TestModeFillInterceptor OnTestModeFill;
public delegate void TestModeUserInterceptor(ref
System.Web.Security.MembershipUser mu);
public event TestModeUserInterceptor OnTestModeGetUser;
public delegate void TestModeUpdateInteceptor(ref object DataTableOrRow);
public event TestModeUpdateInteceptor OnTestModeUpdate;
public void RaiseUserEvent(ref System.Web.Security.MembershipUser mu)
{
    if (OnTestModeGetUser != null) OnTestModeGetUser(ref mu);
}
///
<summary>
    /// Constructor
    /// </summary>
    ///
<param name="ConnectionString">What command to execute</param>
    ///
<param name="ExecuteNonQuery">Whether your command should execute a non query</param>
    public DataExecutor(string CommandString, bool ExecuteNonQuery)
        : this()
    {
        //Connection = new MySqlConnection(str_databaseConnect);
        Command = new MySqlCommand(CommandString, Connection);
        if (!testMode)
        {
            //Connection.Open();
            if (ExecuteNonQuery)
            {
                NonReader = Command.ExecuteNonQuery();
            }
            else
            {
                DataAdapter = new MySqlDataAdapter(CommandString, Connection);
            }
        }
    }
    ///
<summary>

```

```

    /// Initialises a DataExecutor in test mode
    /// </summary>
    ///
    <param name="simulateOnly">Pass this false at your only peril</param>
    public DataExecutor(bool simulateOnly)
    {
        testMode = simulateOnly;
        testExecutor = this;
    }
    ///
    <summary>
    /// Constructor that just opens a connection
    /// </summary>
    public DataExecutor()
    {
        if (!testMode)
        {
            if
            (System.Configuration.ConfigurationManager.ConnectionStrings["ConnString"] != null)
            {
                str_databaseConnect =
                System.Configuration.ConfigurationManager.ConnectionStrings["ConnString"].ToString();
            }
            else
            {
                //hardcode it
                Logging.Logger.Log.Warn("Falling back to hardcoded connection
                string.");
                str_databaseConnect = "CONNSTRING";
            }
            Connection = new MySqlConnection(str_databaseConnect);
            Connection.Open();
        }
    }
    public int Update(genericTableAdapter ata, object UpdateTarget)
    {
        if (!testMode)
        {
            int ret = ata.GenericUpdate(UpdateTarget, this);
            return ret;
        }
        else
        {
            Logging.Logger.Log.Info("DataExecutor running in test mode.");
            if (OnTestModeUpdate != null) OnTestModeUpdate(ref UpdateTarget);
            //We need to work out the object type and return the correct number of
            rows modified
            if (UpdateTarget is DataTable)

```

```

        {
            DataTable dt = (DataTable)UpdateTarget;
            int rowschangedcount = 0;
            foreach (DataRow dr in dt.Rows)
            {
                if (dr.RowState != DataRowState.Unchanged)
                {
                    rowschangedcount++;
                }
            }
            Logging.Logger.Log.Info("Emulating " + rowschangedcount + "
modified rows.");
            return rowschangedcount;
        }
        //It's a DataRow
        Logging.Logger.Log.Info("Emulating 1 modified row.");
        return 1;
    }
}
///
<summary>
    /// Constructor
    /// </summary>
    ///
    <param name="Cmd">What command to execute</param>
    ///
    <param name="ExecuteNonQuery">Whether your command should execute a non query</param>
    public DataExecutor(MySqlCommand Cmd, bool ExecuteNonQuery)
        : this()
    {
        Command = Cmd;
        if (!testMode)
        {
            Command.Connection = Connection;
            if (ExecuteNonQuery)
            {
                NonReader = Command.ExecuteNonQuery();
            }
            else
            {
                DataAdapter = new MySqlDataAdapter(Command);
            }
        }
    }
}
///
<summary>
    /// Execute another command without closing the connection
    /// </summary>

```

```

    ///
    <param name="CommandString">What command to execute</param>
    ///
    <param name="ExecuteNonQuery">Whether your command should execute a non query</param>
    /// <remarks>This function will close the active reader or reset
NonQuery</remarks>
    public void NextCommand(string CommandString, bool ExecuteNonQuery)
    {
        NextCommand(new MySqlCommand(CommandString, Connection), ExecuteNonQuery);
    }
    ///
    <summary>
    /// Execute another command without closing the connection
    /// </summary>
    ///
    <param name="Cmd">What command to execute</param>
    ///
    <param name="ExecuteNonQuery">Whether your command should execute a non query</param>
    /// <remarks>This function will close the active reader or reset
NonQuery</remarks>
    public void NextCommand(MySqlCommand Cmd, bool ExecuteNonQuery)
    {
        if (DataAdapter != null)
        {
            DataAdapter = null;
        }
        if (Command != null)
        {
            Command = null;
        }
        if (NonReader != 0)
        {
            NonReader = 0;
        }
        if (DS != null)
        {
            DS = null;
        }
        Command = Cmd;
        if (testMode) return;
        Command.Connection = Connection;
        if (ExecuteNonQuery)
        {
            NonReader = Command.ExecuteNonQuery();
        }
        else
        {
            DataAdapter = new MySqlDataAdapter(Command);

```

```

    }
}
///
<summary>
    /// The last ID inserted
    /// </summary>
    public int ID
    {
        get
        {
            if (testMode) return 1;
            MySqlCommand LastID = new MySqlCommand("SELECT LAST_INSERT_ID();",
Connection);

            MySqlDataAdapter LastIDDA = new MySqlDataAdapter(LastID);
            DataSet LastIDDS = new DataSet();
            LastIDDA.Fill(LastIDDS);
            return int.Parse(LastIDDS.Tables[0].Rows[0].ItemArray[0].ToString());
        }
    }
    ///
<summary>
    /// Gives a DataSet of the result
    /// </summary>
    public DataSet DataSet
    {
        get
        {
            {
                DS = new DataSet();
                if (testMode)
                {
                    DataTable dt = new DataTable();
                    if (OnTestModeFill != null) OnTestModeFill(ref dt);
                    DS.Tables.Add(dt);
                }
                else
                {
                    DataAdapter.Fill(DS);
                }
                return DS;
            }
        }
    }
    public DataTable DataSetSchema(ref DataTable dt)
    {
        if (testMode)
        {
            //Raise the testmode interceptor event
            if (OnTestModeFill != null) OnTestModeFill(ref dt);
        }
    }

```

```

        else
        {
            DataAdapter.FillSchema(dt, SchemaType.Source);
            DataAdapter.Fill(dt);
        }
        return dt;
    }
    ///
<summary>
    /// Close all objects and dispose of all resources
    /// </summary>
    public void Close()
    {
        if (DataAdapter != null)
        {
            DataAdapter = null;
        }
        if (Connection != null && Connection.State != ConnectionState.Closed)
        {
            Connection.Close();
            Connection = null;
        }
        if (Command != null)
        {
            Command = null;
        }
        if (NonReader != 0)
        {
            NonReader = 0;
        }
        if (DS != null)
        {
            DS = null;
        }
    }
    public static void CopyData(DataTable dtSrc, DataTable dtDest, int limit)
    {
        int counter = 0;
        foreach (DataRow dr in dtSrc.Rows)
        {
            DataRow newRow = dtDest.NewRow();
            newRow.ItemArray = dr.ItemArray;
            dtDest.Rows.Add(newRow);
            counter++;
            if (counter == limit) break;
        }
        dtDest.AcceptChanges();
    }

```



```
}  
}
```

tableAdapter.cs:

```
using System;  
using System.Collections.Generic;  
using System.Text;  
using System.Data;  
using MySql.Data.MySqlClient;  
using System.Reflection;  
namespace Tools  
{  
    public class genericTableAdapter  
    {  
        public int GenericUpdate(object o, DataExecutor de)  
        {  
            if (o != null)  
            {  
                MethodInfo UpdateMethod = this.GetType().GetMethod("Update", new  
Type[1] { o.GetType() }, null);  
                return (int)UpdateMethod.Invoke(this, new object[1] { o });  
            }  
            System.Diagnostics.Trace.TraceInformation("Object was null.");  
            return 0;  
        }  
    }  
}
```